Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier, Julie Bonnaire, and Marie-France Fourcade
Cognitive Neuroimaging Unit
CNRS INSERM CEA

Abstract

We introduce `goxpyriment`, an open-source software framework for programming behavioral and cognitive experiments in the Go programming language. It addresses two recurrent practical difficulties of existing Python-based tools: the runtime-environment complexity that complicates deployment across laboratories, and the unpredictable pauses that automatic memory management can introduce during timing-critical sections. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), `goxpyriment` compiles an entire experiment into a single, self-contained executable with no runtime dependencies, so that distributing an experiment amounts to copying one file. Users can just click on the executable within their file browser and do not need to open a terminal and use the command line (but they can if they want/need to). The programming interface is inspired by Expyriment (Krause & Lindemann, 2014). It provides an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, ...) and audio capabilities (WAV playback and tone generation). For precise timing, response times are obtained by using operating-system event timestamps recorded at hardware-interrupt time rather than by continuous polling, and the garbage collector can be suspended for the duration of a trial. TTL triggers for EEG, MEG and fMRI are supported on a range of devices, and eye trackers (EyeLink, Tobii Pro) are driven through a vendor-neutral interface. We characterize the resulting timing behaviour with the hardware-based protocol of the timing mega-study (Bridges et al., 2020), reporting visual, auditory, and trigger latencies across several hardware configurations. The same source code also compiles to WebAssembly, so that an experiment can be run in a web browser from a URL when browser-grade timing is acceptable. The framework ships with over sixty example experiments and some thirty-five demonstrations that serve as reusable templates for common paradigms; all are available as pre-built programs for Windows, macOS and Linux, and most as browser versions. `goxpyriment` is released under the Apache License 2.0 and is freely available at https://github.com/chrplr/goxpyriment.

Keywords: framework, library, stimulus presentation, timing, behavioral experiments, psychophysics, psychology

Introduction

Experiment-control software provides the foundation for empirical behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording, the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through libraries such as PyEPL (Geller et al., 2007), VisionEgg (Straw, 2008), Expyriment (Krause & Lindemann, 2014) and PsychoPy (Peirce et al., 2019). Before that, MATLAB with the Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) was the de facto standard in many laboratories and is still in use in many of them. More recently, web-based frameworks such as jsPsych (de Leeuw, 2015; de Leeuw et al., 2023) and lab.js (Henninger et al., 2022) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB, extended with the Psychtoolbox, is convenient to generate visual or auditory stimuli and offers exellent timing on well-configured hardware. Yet Psychtoolbox scripts written on Windows most often do not work out of the box on Linux (and vice-versa). Also, MATLAB can be costly for institutions that cannot obtain academic licenses (as is the case for some non-degree-granting research institutions). Python tools provide a more open programming environment with a large scientific ecosystem. Python-based experiments can also achieve high-precision timing, for example by relying on the psychtoolbox Python module which provides pieces of the Psychtoolbox-3 (see https://pypi.org/project/psychtoolbox/). However, the use of Python introduces practical difficulties. For instance, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (pip, venv, conda, uv, ...) and can be a source of breakage across operating-system versions. For example, installing the `psychopy` module under Linux fails while compiling the wxPython package when no precompiled wheel is available. Second, Python's garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording during the critical trial loop. Web-based tools have an obvious advantage in terms of facility of deployment, but browser-mediated stimulus presentation is subject to event-loop latency that makes precise timing difficult (Anwyl-Irvine et al., 2021; Bridges et al., 2020).

The Go programming language (The Go Authors, 2026) offers a different set of tradeoffs. Go is a deliberately small language with a limited feature set and few alternative ways to express a given operation, which tends to make programs easy to read and to reason about (Donovan & Kernighan, 2016). It is statically typed, and its conventions encourage explicit error checking, so that many mistakes surface at compile time rather than at run time, which is a huge time saver for the programmer. Moreover, two properties of Go make it an attractive language to program and distribute experiments. First, Go (cross-)compiles to native machine code and can produce self-contained binaries that embed the stimuli

Christophe Pallier https://orcid.org/0000-0002-3324-666X Correspondence should be addressed to C. Pallier, Cognitive Neuroimaging Unit, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org. I would like to thank Luca Bonatti and Raphael Bordas for alpha testing this software. The authors have no conflicts of interest to disclose.

and experiment lists and have thus no runtime dependencies. Second, its garbage collector is concurrent and can be suspended for the duration of a timing-critical section. These properties motivated our choice of Go as the basis for a behavioral experiment library that prioritizes deployment simplicity and timing control.

The present article describes the design and capabilities of `goxpyriment`, details its timing architecture, and characterizes its temporal precision empirically. We highlight three contributions: (1) zero-dependency deployment via single compiled binaries that simplify distribution across laboratory hardware; (2) a timing architecture that obtains response times from operating-system event timestamps and that can suspend garbage collection during trials, whose behaviour we quantify using the protocol of the timing mega-study (Bridges et al., 2020); (3) a single code base that runs natively on Linux, macOS and Windows and, unchanged, in a web browser through WebAssembly; and (4) a gallery of more than sixty experiments and thirty-five demonstrations, each with its reference and a pre-built binary.

Framework Design

Foundation: SDL3

`Goxpyriment` relies on the Simple DirectMedia Layer, a cross-platform development library designed to provide low-level access to audio, keyboard, mouse, joystick, and graphics hardware (SDL Developers, 2026). SDL has been used to develop thousands of commercial games as well as many emulators. Bindings for many programming languages exist, notably for Python with the `pygame` library (Pygame Developers, 2026). Because of its large user and developer bases, SDL is actively maintained and keeps up with modern hardware. Goxpyriment uses the latest version of this library, SDL3, through the go-sdl3 bindings (Jung, 2026).

Single-Binary Deployment (Zero Dependencies)

Go compiles programs into a single statically linked executable with no external runtime dependencies. Crucially, the `go-sdl3` package embeds the pre-built SDL3 runtime libraries (SDL Developers, 2026) directly into the executable, so that no library needs to be installed separately on the target machine. In addition, the stimuli, experimental lists, fonts, and other assets can be embedded in the binary too. As a result, distributing an experiment requires copying just one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of installing experiments on multiple machines or sharing materials with collaborators.

Another important consequence of the fact that the executable is statically linked is that the user does not have to worry about potential changes in future versions of the library which could impact the behavior of their program. This promotes reproducibility.

Because Go cross-compiles without a C toolchain, every example in the repository is built for Windows (x86-64), macOS (Apple silicon) and Linux (x86-64 and arm64, including the Raspberry Pi) from a single Linux machine by the continuous-integration pipeline, and published both as one archive per platform and as one small archive per experiment (https://downloads.pallier.org/builds/latest/). The Windows builds are linked as GUI

Listing 1: Example of a Parity Decision experiment

```
package main

import (
        "strconv"  // to convert int -> string

        . "github.com/chrplr/goxpyriment/control"
        "github.com/chrplr/goxpyriment/design"
        "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
        exp := NewExperimentFromFlags("Parity Decision",
                Black, White, 32)
        defer exp.End()

        Instructions := "When you'll see a number, your task to decide, " +
                "as quickly as possible, whether it is even or odd.\n\n" +
                "if it is even, press 'F'\n\nif it is odd, press 'J'"

        Targets := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
        NTrialsPerTarget := 2
        trials := design.RepeatList(Targets, NTrialsPerTarget)
        design.ShuffleList(trials)

        EvenResponse := K_F
        OddResponse := K_J
        respKeys := []Keycode{EvenResponse, OddResponse}

        cross := stimuli.NewFixCross(25, 2, DefaultTextColor)

        // creates a map number -> image
        Image := make(map[int]*stimuli.TextLine)
        for _, num := range Targets {
                Image[num] = stimuli.NewTextLine(strconv.Itoa(num),
                        0, 0, DefaultTextColor)
        }

        exp.AddDataVariableNames([]string{"number", "key", "rt",
                "correct"})

        exp.Run(func() error {
                exp.ShowInstructions(Instructions)

                for _, num := range trials {
                        exp.Blank(1500)
                        exp.ShowTimed(cross, 500)
                        key, rt, _ := exp.ShowAndGetRT(Image[num],
                            respKeys, -1)

                        correct := (num%2 == 0) == (key ==
                            EvenResponse)
                        exp.Data.Add(num, key, rt, correct)
                        if !correct {
                                exp.Audio.PlayBuzzer()
                        }
                }
                return EndLoop
        })

        exp.ShowEndMessage("Experiment complete. Thank you!\n\n" +
                "Press any key to exit.")
}
```

programs so that no console window opens beside the experiment. The binaries are not code-signed, so macOS Gatekeeper and Windows SmartScreen show a warning on first launch; the documentation gives the one-line workaround for each.

The same experiment in a web browser

The same source code compiles, unchanged, to WebAssembly (`GOOS=js GOARCH=wasm`). SDL3 and its font, image and audio companions are compiled by Emscripten once and shipped as a pre-built blob in a fork of the `go-sdl3` bindings, so an experimenter needs no Emscripten installation: `make wasm-NAME` produces a directory that any static web server can host. In the browser, the whole `Experiment` API is available: keyboard and mouse responses carry event timestamps, audio plays through Web Audio, and `Screen.Update()` parks on the browser's `requestAnimationFrame`, which is the closest thing a browser has to VSYNC. Session parameters that the desktop version takes from the command line or from the participant-information dialog are read from the URL query string (`?s=3`), and at the end of the session the participant's browser downloads a single `.zip` holding the data CSV and the `-info.txt` companion, byte for byte what a native run writes.

For online data collection, the browser build can also be deployed on a JATOS server (Lange et al., 2015), which hosts the study, distributes links to participants and stores their results. `make jatos-NAME` packages an experiment, again without changes to its source, as a JATOS study archive (`.jzip`) that the experimenter imports through the JATOS web interface. Parameters appended to the study link reach the experiment as they would on the command line, and each call to `exp.Data.Save()` sends the rows recorded so far to the server, so that a participant who closes the tab halfway through leaves behind every block completed until then. The researcher retrieves the same CSV and `-info.txt` files as from a lab session with JATOS's *Export Results*.

Two honest caveats. First, a browser cannot offer the display-stack guarantees of the native build: there is no hardware retrace clock, no real-time priority, and we have not validated pixel onset with a photodiode in a browser. What we did measure is the timestamp resolution and the frame loop: SDL and input-event timestamps tick at about 5 μs when the page is served with cross-origin-isolation headers (about 100 μs otherwise), and a 299-frame presentation loop in headless Chrome ran at 16.666 ms (SD 0.12 ms) with no dropped frame. Paradigms that hinge on sub-millisecond onsets (rapid RSVP, subliminal priming) should therefore run natively; choice reaction-time, memory and attention tasks are well served. Second, some capabilities have no browser counterpart: hardware triggers, gamepads, audio recording, video playback and multi-display fullscreen, and all assets must be embedded (the browser has no file system). Eighty-six of the ninety-eight examples satisfy these constraints and are published as browser versions; each example's README links to its own.

Programming interface: a simple example

Listing listing 1 on the preceding page provides an example of a simple experiment implementing a parity decision task where participants have to decide if a number is even or odd.

- Line 12: An `Experiment` object that manages the window, renderer, and audio subsystems must be created early. Calling it also parses the command-line to get the subject's code and whether the experiment should run in fullscreen mode or not.
- Lines 16–36: This part of the code prepares the stimuli. Lines 31–36 are the most Go-specific: they create a map (a.k.a. associative array or dictionary in other programming languages) associating each number to a graphic texture. By pre-building these textures, we ensure that they will be displayed as quickly as possible in the presentation loop (we could have called `stimuli.NewTextLine()` in the loop, saving a few lines at the cost of recomputing the texture at each iteration).
- Lines 41–56: This part of the code deals with the presentation loop. It is wrapped in a `Run` call that catches quitting events: if the participant presses the Escape key or closes the window, the library catches the resulting internal signal, safely flushes the data file to disk, and shuts down the SDL subsystems, ensuring that data is preserved even if a session is aborted prematurely. Line 47, the function `ShowAndGetRT()` presents the visual stimulus, waits for one of the given keys, then returns the reaction-time.

Timestamps, Garbage Collection Control, ...

Response timing in `goxpyriment` exploits SDL3 OS-level event timestamps. The `ShowAndGetRT` function used in the parity example is a wrapper for:

```
onset,_ := exp.ShowTS(stim)
key,eventTS,_ := exp.Keyboard.GetKeyEventTS(keys,
                                            timeoutMs)
rt := int64(eventTS - onset) / 1_000_000
```

`ShowTS()` displays the stimulus `stim`, blocking until the next screen flip when VSYNC is enabled, and returning a time-stamp for that event (in ns). When VSYNC is disabled, `ShowTS()` returns immediately, supporting variable refresh rate (VRR) monitors.

`GetKeyEventTS()` scans the keyboard and returns the code of the key pressed and a timestamp (in ns) for it. This timestamp is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). Because SDL events carry their timestamp (information that pygame discards), the event remains in the queue with its original timestamp regardless of how much computation occurs between stimulus onset and response collection. By comparing the response timestamp with the `ShowTS()` timestamp, reaction times can be computed without interpreter overhead (hardware-level lags at the keyboard or mouse can, of course, still be present). An advantage of this approach is that there is no need to poll the HID device continuously to detect events: a complex sequence of stimuli can be programmed and every event remains available, with its original timestamp, to be processed later when needed.

To reduce the likelihood of latencies, goxpyriment provides additional functions:

Explicit Garbage Collection Control Go, like Python or JavaScript, has a memory garbage collector. This can introduce unpredictable pauses that corrupt timing. However, unlike Python or Javascript, Go allows the programmer to suspend the garbage collector. This can be used in `goxpyriment`'s timing-critical loops to prevent garbage-collection during a trial.

High-Precision Streams For paradigms with dense, high-speed stimulus sequences (e.g., retinotopic stimulation), `goxpyriment` provides specialized stream functions such as `PresentStreamOfImages`. These functions disable garbage collection, drain the event queue, and busy-wait at the sub-frame level to ensure that each stimulus onset is scheduled accurately. A `TimingLog` records the target and actual onsets for post-hoc verification, as well as the timing of keys pressed during the presentation of the stream.

Real-Time Scheduling Priority On Linux, an experiment built with `goxpyriment` requests the `SCHED_FIFO` real-time scheduling policy for itself at start-up (priority 50 by default), rather than relying on the user to remember a `chrt` prefix on the command line — which is in any case impossible when the program is launched by clicking its icon. The request is made with `sched_setscheduler()` on the main thread, to which the Go runtime has been pinned, so that the elevation applies to the thread that runs the trial loop. Under the default (`SCHED_OTHER`) policy the kernel may deschedule the process at any point, including between a stimulus onset and the trigger that marks it; this is the dominant source of the rare, large latencies that otherwise survive the measures above.

The request is not silently mandatory. It requires a system-level grant (an `rtprio` limit for the user, set in `/etc/security/limits.d/`), and if that grant is absent the program reports the reason and continues at normal priority rather than refusing to run. It can be disabled with the `-no-realtime` flag — useful when debugging, since a breakpoint hit on a real-time thread can leave the desktop unresponsive — and the priority can be changed with `-realtime-priority`. The policy actually obtained is recorded in the session's system report, so that a data set carries evidence of the conditions under which it was collected rather than of the conditions that were intended.

Triggers

To communicate with external recording equipment, such as fMRI, EEG, and MEG systems, `Goxpyriment` exposes every TTL device through one eight-line input/output interface, so that an experiment written against one device runs against another by changing a flag. The current version drives the parallel port and GPIO lines (Linux), the DLP-IO8 (https://github.com/chrplr/dlp-io8-g) and DLP-IO20 USB-serial boxes, the Adafruit FT232H (in pure Go, without the vendor library), the LabJack T4 over Ethernet (https://labjack.com/products/labjack-t4, without its SDK), the NEUROSPEC MMBT-S, and an Arduino Mega 2560 running a custom firmware that also timestamps button presses on the device (https://github.com/chrplr/neurospin-meg-ttl-box). Beyond TTL, the package talks to a generic serial port, to EGI NetStation over its network protocol, and to a networked video recorder. Adding support for new hardware is facilitated by the availability of the source code for the current devices.

How the pulse is issued matters as much as which device issues it. Go makes asynchronous execution trivial—prepending the `go` keyword to a call runs it concurrently and returns at once—and it is tempting to fire every trigger that way. But a goroutine is scheduled at the runtime's convenience: measured against the flip timestamp, a rising edge dispatched through a goroutine at normal priority under load arrived 0.73 ms late with

about 1 ms of spread, while the same edge raised synchronously on the statement after the flip, at real-time priority, arrived at a median of 34 μs (maximum 37 μs). `goxpyriment` therefore provides `FireTriggerSync`, which raises the line on the calling thread and defers only the falling edge, which carries no information, to a goroutine. The fire-and-forget form remains the right one for a block marker or an end-of-run flag, where the edge is not the measurement.

The device itself costs little. Firing the same pulse train on two devices from one schedule and reading the edges on an oscilloscope, the DLP-IO8's edge lagged the FT232H's by 179.4 μs (SD 5.7, $n = 322$) on one machine and the parallel port's by 185.9 μs (SD 12.6, $n = 900$, worst pulse 254.8 μs) on another: the USB-serial path adds a constant of about 180 μs and a jitter an order of magnitude below a millisecond. (This is the delay between two devices on one schedule; the delay of a device's edge relative to the flip timestamp is reported separately in the Supplementary Material.)

Eye tracking

Eye trackers are reached through a vendor-neutral `Tracker` interface—open, calibrate, start and stop recording, mark, read the latest sample or drain the buffered samples and the tracker's own fixation, saccade and blink events—with implementations for the SR Research EyeLink and the Tobii Pro range, a null tracker so that an experiment runs unchanged without hardware, and a simulated tracker driven by the mouse for development. Both vendors publish their interface only as a C library or a native Python extension, and linking either would cost `goxpyriment` its pure-Go build, its cross-compilation and its browser target. The vendor SDK therefore runs in a small Python *bridge* process that the experiment talks to over a loopback TCP socket, one JSON object per line. This is the shape PsychoPy's ioHub arrived at for a different reason: there the separate process keeps the interpreter lock out of the sampling loop; here it keeps the C compiler out of the build. Supporting a further tracker is a new bridge script, not a new Go package.

Calibration is one call, `exp.CalibrateTracker`, behind which two procedures hide. The EyeLink draws its own calibration targets in a window it owns, after which `goxpyriment` reclaims the display and keyboard focus; the Tobii SDK draws nothing, so `goxpyriment` draws the targets itself, with flip-timestamped onsets, and drives the tracker's collect-and-compute protocol point by point. The outcome of either is written to the session's `-info.txt`.

The bridge is deliberately kept out of the timing path. Sending a message through it and waiting for the tracker's acknowledgement took a median of 600–719 μs and up to 1.2 ms over three runs on an EyeLink at 2000 Hz—small against a frame, but not a way to time a stimulus. Stimulus onsets are therefore marked by a TTL edge into the tracker's digital input (with `FireTriggerSync`, as above), and bridge messages serve only to label trials. The relation between the tracker's clock and the experiment's is measured NTP-style before and after each run, and both the offset, with its round-trip uncertainty, and the drift in parts per million are recorded in the data file's companion.

Miscellaneous additional features

This parity decision example only demonstrates a tiny part of the features provided by the framework.

For example, it is possible to build the experiment around a hierarchical structure of blocks and trials, over which the program can iterate, following a pattern inspired by Expyriment (Krause & Lindemann, 2014). Each trial is a collection of factors that can be randomized and logged. Data collection is handled by an `exp.Data` object, which writes *two* files per session. The first is the trial data as plain CSV, with no comment or metadata lines, so it opens directly in a spreadsheet program or a statistics package without preprocessing. The second is a companion `-info.txt` holding everything about the conditions the session ran under: the machine, operating system, kernel and desktop environment; every GPU and its driver; the sound server and its version; the display mode, audio device and buffer size that were actually opened; the scheduling policy obtained; and the participant details. Splitting them this way means the analysable file needs no cleaning and the provenance is still recorded — and recorded automatically, rather than depending on someone writing it down. An example appears in the Supplementary Material (Figure A2).

Beyond the stimuli visible in the example, the library provides text-entry and menu widgets, thick poly-lines and filled polygons, sprite sheets, random-dot stereograms, Gabor patches, dot clouds and VSYNC-locked animation loops (drifting gratings, moving dot clouds), and the *stream* functions for RSVP and mixed audio-visual sequences. Video plays in a frame-exact format (`.gv`) whose onsets are locked to the retrace, and a `media` package schedules several movies on one master clock. Naming latencies are measured with a software voice key on the microphone input, which returns the vocal onset timestamp and saves the recording per trial. For psychophysics, a `staircase` package implements the up-down (Levitt, 1971) and QUEST (Watson & Pelli, 1983) procedures, and a `units` package converts between pixels, centimetres and degrees of visual angle for a described monitor and viewing distance.

A small but consequential safeguard: modern desktops refuse to give a window keyboard focus that it did not obtain from the user, and SDL delivers key events only to the focused window, so an experiment started from a script, or resumed after an eye-tracker calibration owned the display, could run its trials without seeing a single response. `goxpyriment` detects this, flashes its window and waits for the click before the first trial.

Another thing which is not transparent from the source code is the fact that `goxpyriment` provides a graphical `GetParticipantInfo` dialog that opens (optionally) before the main experiment window (see Fig. 1). This allows the experimenter to input subject IDs, select experimental conditions from discrete menus, and toggle settings like fullscreen mode. These values are automatically recorded in the output data file header.

Timing Validation

We follow the hardware-measurement protocol of the timing mega-study (Bridges et al., 2020): external instruments record the physical events—light from a photodiode on the screen, sound from the line output, TTL edges on a digital line—independently of the computer under test, so the reported latencies do not depend on the timing of the machine being measured. Alongside the Black Box Toolkit (BBTK) we recorded the same events

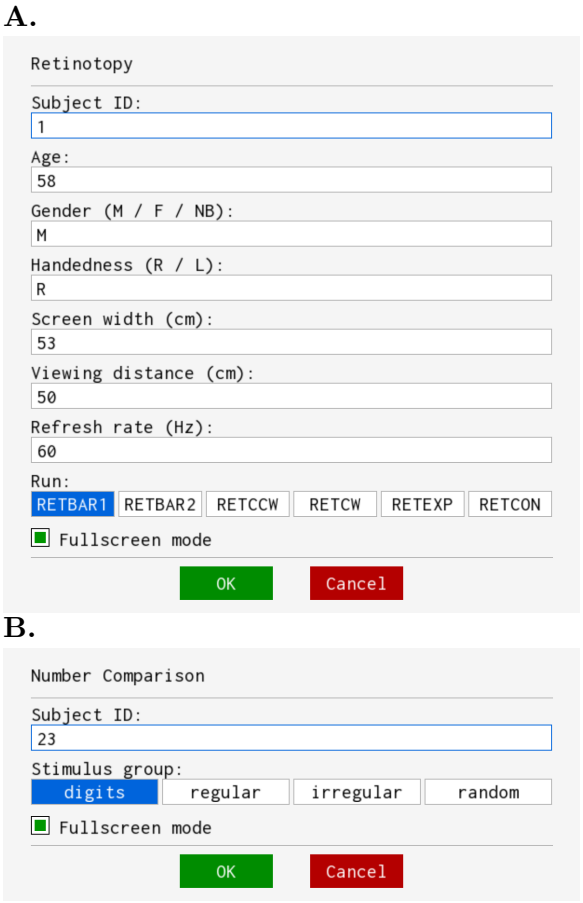


Figure 1

Examples of `GetParticipantInfo` *dialogs for* Retinotopy *(A.) and* Number-Comparison *(B.), two paradigms provided in the examples accompanying the library. These UIs makes it practical for the experimenter to choose among a small set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager. Note that which fields appear in the UI is configurable. It is also possible to pass the information as arguments on the command-line and skip these UIs.*

with a Digilent Analog Discovery 3 (AD3) sampling two analogue channels at 100 kS/s on a single timebase. The BBTK quantises onsets to 0.25 ms and applies a detection threshold it does not report; the AD3 stores the waveform, so the onset criterion is chosen at analysis time and stated with the result.

Three questions organise what follows, in the order an experimenter should ask them. Are the frames presented at all, and for as long as requested? How stable is the delay from a trigger to the light? And where does the sound fall relative to the image? The apparatus, the individual runs, the commands that produced every number, and the full tables and supporting analyses are given in the Supplementary Material.

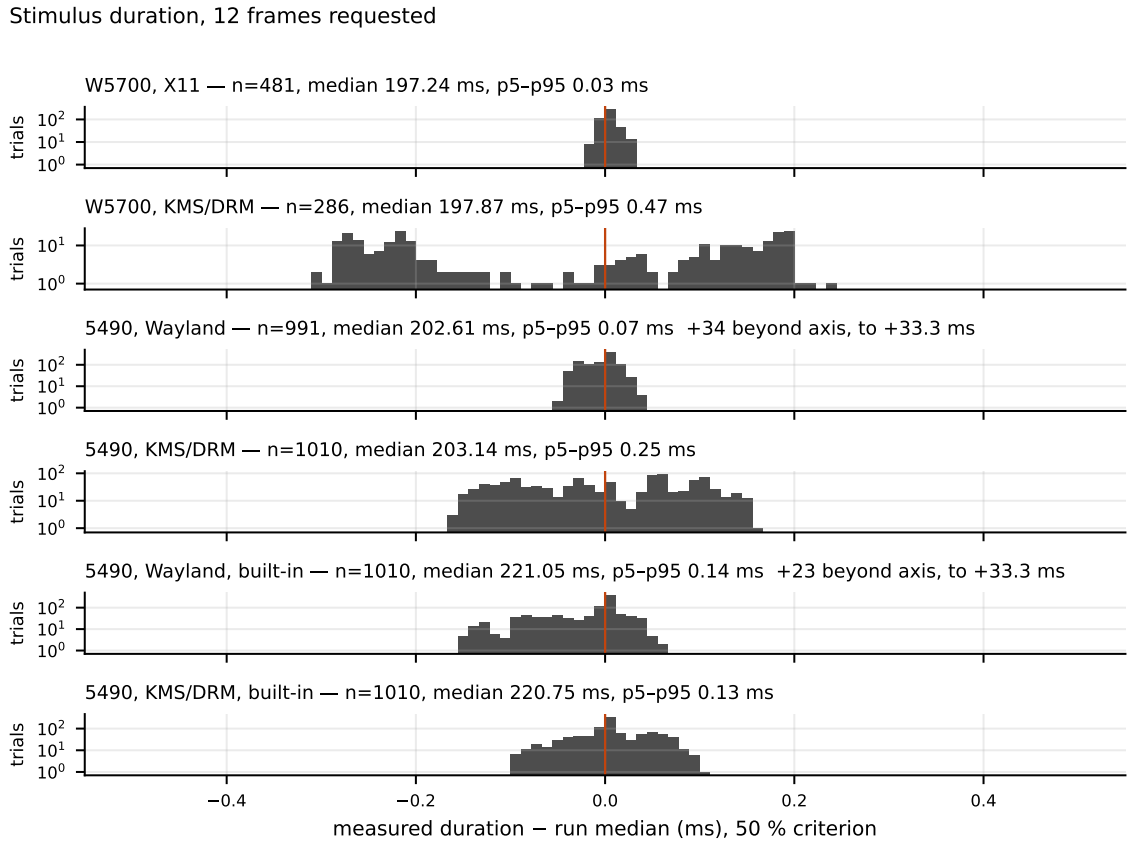


Figure 2

Optically measured duration of a 12-frame stimulus, photodiode on the screen, 50 % criterion on both edges, plotted as deviation from each run's own median. Counts are logarithmic so that a handful of long trials remains visible against a mode of several hundred. Trials falling outside the axis are counted in each panel title; they are dropped or held frames, not scatter. Absolute medians differ by panel and by machine and are not comparable across rows—see the Supplementary Material.

Are frames dropped, and are durations what was asked for?

Figure 2 shows the optically measured duration of a nominal 200 ms (12-frame) stimulus presented in a 500 ms cycle, on three machines under four display stacks. Each panel is drawn as deviation from its own median, because the absolute value is a property of the panel and not of the software: the same monitor driven by two of our machines returned medians 5.9 ms apart, which liquid-crystal rise/fall asymmetry accounts for almost entirely. What compares across configurations is the width of the bulk and the size of the tail.

Where no frame is dropped, presentation is exact: the middle 90 % of trials spans 0.03–0.47 ms, individual durations repeat to 11–91 μs, and the 30-frame cycle period matches nominal to within 9 μs. The tail is what separates the configurations, and one factor produces all of it. Under a Wayland compositor 2.3–3.4 % of stimuli ran long—by

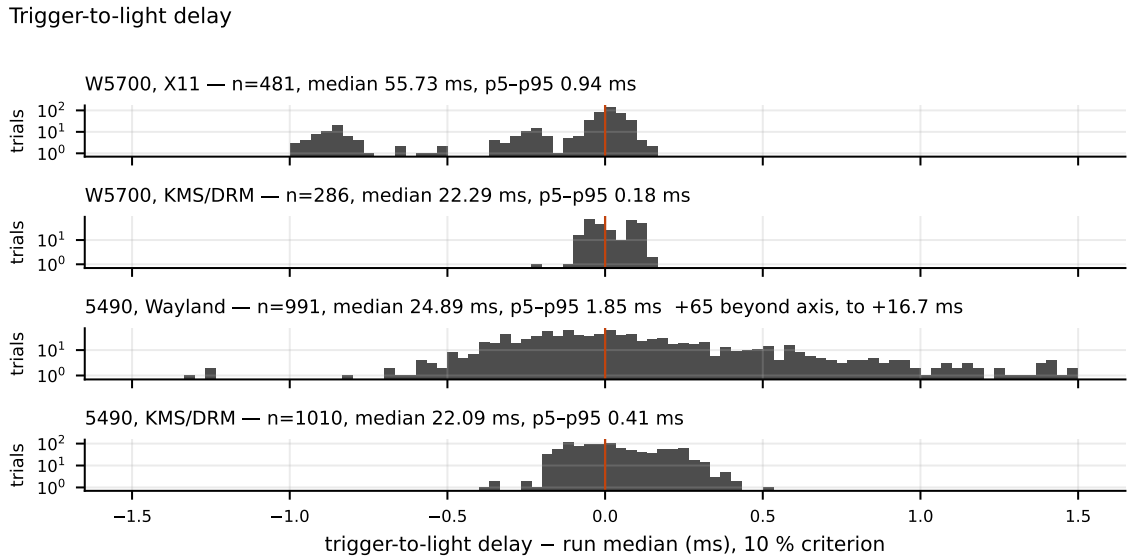


Figure 3

Trigger-to-light delay, deviation from each run's median, 10 % criterion, logarithmic counts. Median delays were 55.7, 22.3, 24.9 and 22.1 ms respectively: a display server costs about two frames of latency, but it is the width that separates the configurations. Absolute values and the flip-to-photon figures are tabulated in the Supplementary Material.

one frame usually, by 33 ms at worst—and the same runs dropped 15 and 43 frames per thousand cycles, against zero to one everywhere else. No host-side statistic saw any of this: those runs report that the driver blocks, sub-ppm stability against the panel, and frame intervals with sub-millisecond scatter. Only the photodiode showed it.

This is also why we report the tail rather than a standard deviation. In these data an SD of 3.654 ms belonged to a distribution whose middle 90 % spanned 0.08 ms; the SD describes neither the bulk nor the failure.

How stable is the trigger-to-light delay?

Two quantities matter here. Trigger-to-light is what an EEG or MEG analysis needs in order to place a stimulus in a recording; flip-timestamp-to-photons is what the framework told the experiment against what the participant saw, and it is the one that enters a single-trial reaction time. Figure 3 shows the first, per trial, on the same four configurations.

The timestamps track the display. A stimulus generator reports an onset and reaction times are measured from it; if that time advances on a clock other than the display's, the error grows for as long as the block lasts. An earlier version of this framework held each frame to a schedule derived from the nominal refresh rate, and on one machine the reported onsets walked away from the photons at −4.73 ppm—2.3 ms per eight-minute block, exceeding a frame within the hour—while its own frame-interval statistics stayed immaculate. That machine's GPU and CPU keep independent crystals, and its loop cadence

differs from nominal by −4.20 ppm: the crystal difference *was* the drift. Anchoring each onset on the instant `SDL_RenderPresent` returns, wherever the driver blocks on the retrace, re-establishes the phase from hardware every frame. Measured against a photodiode over 1010 cycles per run, the residual drift is +0.01, +0.13 and −0.77 ppm on three machines—0.01 to 0.4 ms over an eight-minute block—while the same machine's cadence remains −4.20 ppm from nominal. That the cadence and the drift now disagree by a factor of thirty is the evidence that onsets follow the panel and not the schedule.

Precision is set by the display stack, not by the trigger device. Under a compositor the flip-to-photon scatter is 0.674 ms; on the same machine, panel, photodiode and trigger, running on bare KMS/DRM it is 0.079 ms—an order of magnitude, from a setting. The trigger device is not the limitation it is assumed to be: a USB-serial trigger under KMS/DRM (0.057 ms) beat a directly-driven GPIO line on another machine (0.075 ms), and both trigger boxes put their edge out within 20 μs of the flip.

Where does the sound fall?

Audio was characterised on two machines by capturing the line output electrically rather than through a microphone. The audio-visual lag was 49.88 ms (SD 6.17 ms) on a Raspberry Pi's on-board codec and 23.34 ms (SD 6.54 ms) on a USB audio interface, both at a 1024-frame buffer. *The third machine has not been measured for audio*; that figure is missing rather than assumed.

The scatter is not device noise. The tone is handed to the device on time—the software-side interval between the visual flip and the moment the tone was queued was 0.080 ± 0.035 ms—and then waits for the queue, so its onset falls uniformly across one buffer period. Figure 4 shows measured scatter against the period/$\sqrt{12}$ this predicts: they agree to within 8 % at every buffer size, on both machines. Below 1024 frames the tones tore audibly on our hardware. Buffer size therefore sets audio latency and audio jitter together, and it is the single most consequential audio setting an experimenter chooses.

Recommendations

- Measure the configuration, not the machine. The largest effects we found were settings: 32 ms of latency from a display server, an order of magnitude of onset precision from a compositor, 79 ms and four times the jitter from an audio buffer.
- If absolute onset latency or single-trial precision matters, run without a display server.
- On Wayland, run the panel at its native resolution: with a scaled desktop the refresh rate reported to the program was wrong by 1477 ppm, and no program can detect this from the inside.
- Choose the audio buffer by measurement—as small as the machine sustains without tearing—and make the trial cycle a whole multiple of it.
- Warm the display in its final mode before recording; liquid-crystal response and panel rate both move while a display settles.
- State the onset criterion with any latency, the panel with any duration, and the buffer size with any audio figure. Report the count of trials outside the bulk, not only a standard deviation.

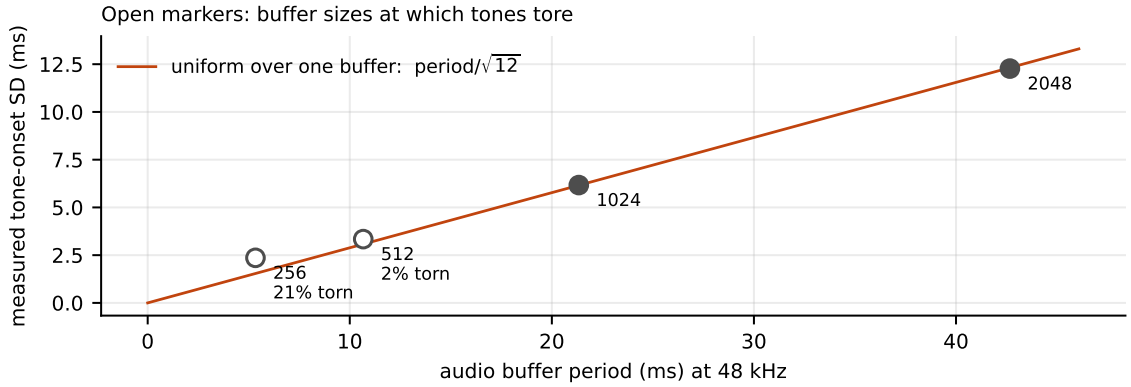


Figure 4

Measured tone-onset scatter against audio buffer period, with the scatter predicted if the onset falls uniformly within one buffer. Both latency and jitter scale with the buffer, and below 1024 frames this machine could not deliver tones intact. The scatter is a deterministic beat rather than noise, and can be removed entirely by choosing the trial cycle as a whole multiple of the buffer period (Supplementary Material).

The suite that produces these measurements ships with the framework (`tests/Timing-Tests/`) and writes the configuration—display mode, audio device and buffer, scheduling policy, onset source, and the pacing branch each frame took—into every data file, so that a capture carries the conditions it was collected under.

Gallery of Examples

The repository carries, at the time of writing, 63 experiments, 35 demonstrations and 26 hardware tests, catalogued at https://chrplr.github.io/goxpyriment/GalleryOfExamples/. An *experiment* implements a published paradigm and records behavioural data; a *demonstration* shows one feature (an illusion, a widget, a stream) and measures nothing; a *test* is a program whose output is analysed to check the performance of the machine it runs on (the timing suite of Section , the trigger-device comparison, the eye-tracker checks). Every directory declares its category, a one-line description and its literature reference in a small metadata file from which the gallery is generated, and carries a README.

The experiments span the classic repertoire of the field: attention (Posner cueing and the ANT, attentional blink, change blindness, multiple-object tracking, visual search, the psychological refractory period); memory (Sperling's iconic memory, Sternberg scanning, memory span, memory for binary sequences); numerical cognition (SNARC, number comparison, number change detection, magnitude estimation); language and reading (lexical decision, masked priming, the letter-size illusion, temporal integration in word recognition, Stroop and picture naming with a voice key, shadowing); geometry and reasoning (intruder and triangle-completion tasks, mental rotation in two and three dimensions, the ConceptARC benchmark, Rush Hour); psychophysics (contrast detection by QUEST, auditory threshold estimation); statistical learning (auditory, visual, community structure); infant looking-time paradigms; motor timing (finger tapping and tracking, mouse tracking);

Table 1

Comparison of selected experiment frameworks.

Property	goxpyriment	Expyriment	PsychoPy	PTB (MATLAB)	jsPsych
Language	Go	Python	Python	MATLAB	JavaScript
Deployment	Single binary	pip/conda	Standalone or pip	MATLAB license	CDN / npm
GC control	Explicit disable	No	No	N/A	No
VSYNC sync	Yes (SDL3)	Yes (SDL2)	Yes (Pyglet/PTB)	Yes	No
Hardware triggers	Yes	Yes	Yes	Yes	No
Eye tracking	EyeLink, Tobii (bridge)	No	ioHub (several vendors)	EyeLink toolbox	WebGazer (webcam)
Response device API	Unified	Device-specific	Device-specific	Device-specific	N/A
Event timestamp	SDL3 hardware	pygame timer	iohub/PTB clock	`GetSecs` (μs)	Browser clock
Stimulus set	Moderate	Moderate	Extensive	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No	No
Online deployment	Yes (same source, WASM)	No	Partial (PsychoJS)	No	Yes
Reference	This paper	Krause and Lindemann, 2014	Peirce et al., 2019	Brainard, 1997	de Leeuw et al., 2023

and neuroimaging localizers (language localizers in three variants, MEG localizers, standard and multifocal retinotopic mapping, RSVP of images and of mathematics and language). Each serves as a functional template, and the collection as a whole doubles as a corpus from which AI coding assistants generate new experiments in the same idiom.

All examples and demonstrations are available pre-built for Windows, macOS and Linux (Section), and most also run in the browser (Section). Building them locally takes three steps: install Go, clone the repository, and run `examples/build.sh`.

Comparison with Existing Tools

Table 1 summarizes key properties of `goxpyriment` alongside five other experiment frameworks.

`goxpyriment` resembles Expyriment in its API design: a central experiment object, a `present()`-based stimulus API, a structured trial-loop model, and output to `.csv`-format files with metadata. The code for a simple experiment (Parity Decision) is provided in Listing 1. The primary technical differences are the use of Go rather than Python and the use of SDL3 rather than the older Pygame/SDL2 stack. From the user's point of view, `goxpyriment` adds *stream* objects which, as their name indicates, allow displaying sequences of stimuli with tightly controlled timing.

Table 1 necessarily simplifies; each framework has strengths that a compact matrix cannot convey. PsychoPy, in particular, offers a graphical Builder interface, a larger and

extensively documented stimulus set, mature online deployment through PsychoJS, and a large user community. The Psychtoolbox remains a reference for low-level timing control on well-configured hardware. Relative to these established tools, `goxpyriment` is very young: it has no graphical builder and a more limited stimulus library, and its browser deployment, while it runs the same source, carries neither the hardware retrace clock nor the photodiode validation of the native build. Its distinctive contributions are therefore deployment simplicity—one binary, or one URL, from one source—and an explicit timing architecture rather than breadth of features. The entries in Table 1 describe design characteristics; measured timing comparisons on identical hardware are reported in Section .

Expandability

The availability of the source code makes it easy to expand goxpyriment, e.g., adding support for new hardware. Given a clear specification for an hardware interface, an human, or an AI coding assistant can quickly generate Go code for this hardware taking example from the existing packages.

Goxpyriment is distributed under an Apache-2 license, a free software license compatible with the GNU GPLv3 according to the Free Software Foundation (GPLv3 projects can include code licensed under Apache-2).

Limitations and Future Directions

Timing is characterised on Linux only. Every figure in Section was measured under Linux. The framework builds and runs on macOS and Windows, but real-time scheduling is requested only on Linux, and no photodiode measurement has been made on the other two systems; an experimenter who needs onset guarantees there should run the bundled timing suite on their own machine before trusting any number.

Browser onsets are unvalidated. The browser build has been checked for timestamp resolution and frame pacing but not, so far, against a photodiode; until it is, it should be reserved for paradigms that tolerate browser-grade onset precision.

Eye-tracker calibration. On the EyeLink, calibration targets are drawn by the vendor library in its own window rather than by `goxpyriment`, so the calibration display cannot yet be styled like the experiment; the Tobii path already draws its own targets and the same could be done for the EyeLink. Trackers that publish a network protocol (GazePoint, Pupil Labs) could be supported by a pure-Go client without a bridge.

No video recording. While there is currently some support for recording audio responses and measuring naming times, the framework does not currently allow to capture videos (unless one controls a videorecorder through the serial interface). As SDL3 supports recording video, this feature might be implemented in the future.

No way of sending results to a web server. Currently, results files are saved locally. In some situations, it could be useful to send the (anonymized) results to an on-line database, for example using a REST api.

Language barrier. Researchers who are unfamiliar with Go, that is, the vast majority, will need to invest a bit of time in getting acquainted with the language. To this end, many web resources exist, for example, https://go.dev/tour/ and https://gobyexample.com/. As Go is a descendant of C, the syntax will be familiar to people who

know a language of the same family, e.g., Java or Javascript. Finally, the examples and the migration guide we provide at https://chrplr.github.io/goxpyriment, are also meant to ease the transition. AI-Coding tools can help a lot in porting existing matlab or python code, adapting existing experiments, or creating new ones.

Declarations

Funding

No funding was received to assist with this project nor with the preparation of this manuscript.

Conflict of interest/Competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

The raw instrument captures and run logs behind the Timing Validation section, and the scripts that turn them into the reported figures, are available with the source repository (see the Supplementary Material).

Code availability

The source code of `goxpyriment` is available under the Apache License 2.0 at https://github.com/chrplr/goxpyriment. All example experiments described in this article are included in the repository. No behavioural data from human participants are reported; the empirical measurements in this article are hardware timing measurements.

Authors Contributions

Christophe Pallier: Conceptualization, Software implementation, Documentation and Paper Writing, with assistance from **Claude Opus 5** and **Gemini 2.5**. Consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude and Gemini are listed under Author Contributions rather than as a named author, because authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content. **Julie Bonnaire** and **Marie-France Fourcade** set up the timing measurement system and performed tests.

References

Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, *52*(1), 388–407. https://doi.org/10.3758/s13428-019-01237-x

Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, *10*(4), 433–436. https://doi.org/10.1163/156856897X00357

Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, *8*, e9414. https://doi.org/10.7717/peerj.9414

de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, *47*(1), 1–12. https://doi.org/10.3758/s13428-014-0458-y

de Leeuw, J. R., Gilbert, R. A., & Luchterhandt, B. (2023). jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, *8*(85), 5351. https://doi.org/10.21105/joss.05351

Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.

Geller, A. S., Schleifer, I. K., Sederberg, P. B., Jacobs, J., & Kahana, M. J. (2007). PyEPL: A cross-platform experiment-programming library. *Behavior Research Methods*, *39*(4), 950–958. https://doi.org/10.3758/BF03192990

Henninger, F., Shevchenko, Y., Meijer, R. R., Inzlicht, M., & Hilbig, B. E. (2022). Lab.js: A free, open, online study builder. *Behavior Research Methods*, *54*(2), 556–573. https://doi.org/10.3758/s13428-019-01283-5

Jung, B. (2026). *SDL3 Bindings for Go*. Retrieved April 1, 2026, from https://github.com/Zyko0/go-sdl3

Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What's new in Psychtoolbox-3? *Perception*, *36*(ECVP Abstract Supplement), 14. https://pure.mpg.de/rest/items/item_1790332/component/file_3136265/content

Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, *46*(2), 416–428. https://doi.org/10.3758/s13428-013-0390-6

Lange, K., Kühn, S., & Filevich, E. (2015). "Just Another Tool for Online Studies" (JATOS): An Easy Solution for Setup and Management of Web Servers Supporting Online Studies. *PLOS ONE*, *10*(6), e0130834. https://doi.org/10.1371/journal.pone.0130834

Levitt, H. (1971). Transformed up-down methods in psychoacoustics. *The Journal of the Acoustical Society of America*, *49*(2B), 467–477. https://doi.org/10.1121/1.1912375

Nature Portfolio. (2023). Tools and technologies: Artificial intelligence.

Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

Pygame Developers. (2026). *Pygame: Python game development development library*. Pygame Community. Retrieved July 26, 2026, from https://www.pygame.org/

SDL Developers. (2026). *Simple DirectMedia Layer, sdl3*. Retrieved April 1, 2026, from https://www.libsdl.org/

Springer Nature. (2023). Artificial intelligence (AI) policy for authors.

Straw, A. D. (2008). Vision Egg: An open-source library for realtime visual stimulus generation. *Frontiers in Neuroinformatics*, *2*. https://doi.org/10.3389/neuro.11.004.2008

The Go Authors. (2026). *Go developpers website*. Google. Retrieved July 26, 2026, from https://go.dev

Watson, A. B., & Pelli, D. G. (1983). QUEST: A Bayesian adaptive psychometric method. *Perception & Psychophysics*, *33*(2), 113–120. https://doi.org/10.3758/BF03202828

Appendix
Supplementary Material: Timing Validation in Detail

This appendix documents the apparatus, the individual runs, the commands that produced every number in the Timing Validation section, and the analyses that the main text summarises. All measurements were made on 2026-08-16 to 2026-08-18, and each is traceable to a capture file.

Apparatus

Machines. Three, chosen to span the range of hardware a laboratory is likely to own rather than to represent it: a single-board computer, a workstation with a professional GPU, and a current laptop. All ran Linux with SDL 3.4.0, with the garbage collector suspended for the duration of each measurement loop and the process at `SCHED_FIFO` priority 50.

Host	Machine	Graphics	Audio
`omv-pi4`	Raspberry Pi 4B	V3D 4.2.14.0	on-board codec
`pcstim7`	workstation	AMD Radeon Pro W5700 (radeonsi, navi10)	Focusrite Scarlett Solo 3rd Gen
`is158520`	Dell Precision 5490	Intel Arc Graphics (Meteor Lake)	on-board (PipeWire)

Displays. A single Dell 1905FP (1280×1024, nominal 60.0197 Hz) was shared between `pcstim7` and `is158520`, connected by a DisplayPort-to-HDMI adapter on the first and a USB-C-to-HDMI hub on the second. The 5490's built-in panel (2560×1600) was measured separately, and the Pi drove an HDMI display. The same physical monitor on two machines is what allows the panel contribution to be separated from the software's (*What the panel contributes*, below).

Instruments. A Black Box Toolkit, and a Digilent Analog Discovery 3 sampling two channels at 100 kS/s on one timebase. Channel 1 carried a photodiode taped to the screen over the stimulus square; channel 2 carried the trigger line, or the audio line output for the audio-visual runs. Because both channels share one acquisition and one clock, the instrument's timebase cancels in any interval measured between them.

Two properties of the AD3 rig cost us time and are worth stating. Its analogue inputs are differential, so a lead that leaves the negative input floating reads mains hum rather than signal; and the device must be on a root USB port—behind a laptop dock it enumerates normally, accepts every configuration, and then delivers about 20 samples per second before stalling.

Photodiode. Figure A1 gives the detector circuit. It is a passive design on purpose: with no amplifier in the path there is no amplifier settling time to disentangle from the panel's. The load resistance and the junction-plus-cable capacitance, tens of picofarads, put the detector's own time constant well under a microsecond—below the 10 μs sample interval, and three orders of magnitude faster than the millisecond panel transitions it is used to measure.

Onset and offset criteria are applied per trial against that trial's own dark and bright levels rather than against a fixed voltage, so a slow drift in backlight output does not leak into the transition times. Levels were stable throughout: the dark level sat at 0.15–0.25 V and the bright level at 8.18–8.19 V across every session reported here, which

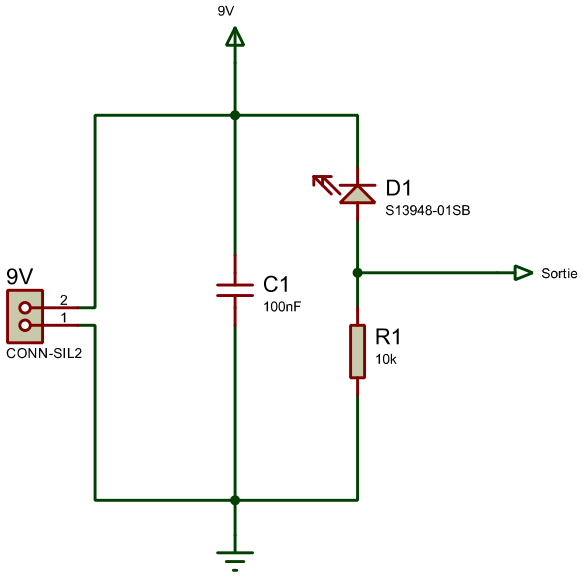


Figure A1

The photodiode detector used on channel 1. D1 is reverse-biased from a 9 V battery and its photocurrent develops across the 10 kΩ load R1, so the output at Sortie *rises with screen luminance; C1 decouples the supply. Running the detector from a battery keeps it off the mains ground of the machine under test.*

is also what rules out a difference in signal range as the explanation for the two 1905FP sessions disagreeing (*What the panel contributes*, below).

Triggers. A DLP-IO8 USB relay box on `pcstim7` and `is158520`, and a directly-driven GPIO line on the Pi. Measured against the flip timestamp, both put their edge out within 20 μs (*Onset timing*, below), so neither limits the measurements reported here.

Runs and commands

Each run presents a square for 12 frames in a 30-frame (500 ms) cycle, emitting a TTL pulse on the flip that turns it on. The commands below are those recorded in each run's own `-info.txt`; the AD3 capture is started first, from a second machine or a second terminal, and outlives the run.

```
# 1. start the capture (root USB port; --save-raw is needed for envelope work)
ad3-capture --seconds 545 --rate 1e5 --channels 1,2 --save-raw --out run.npz

# 2. run the measurement (examples as actually invoked)
Timing-Tests -test display -duration-s 300
Timing-Tests -test av -cycles 1010 -no-sound -trigger-ms 15 \
             -trigger-device dlpio8 -port /dev/serial/by-id/usb-DLP_Design_DLP-IO8_
                 ...
Timing-Tests -test av -cycles 1010 -no-sound -trigger-ms 15 \
             -trigger-device dlpio8 -d 1 -port /dev/serial/by-id/...   # external
                 display
```

```
Timing-Tests -test av -cycles 500 -audio-frames 1024 -trigger-device dlpio8 -trigger-
    ms 15

# 3. extract events, and fit the host-to-instrument clock relation
./cmd/ad3-to-events.py run.npz events.csv       # also prints the panel rise/fall
./_build/timing-drift events.csv Timing-Tests_sub-000_date-....csv

# 4. regenerate the figures in this paper
./cmd/plot-timing-figures.py <capture-dir> paper/figures
```

To change the display stack, the desktop session was left running for the Wayland and X11 conditions, and stopped in favour of a virtual console for the KMS/DRM conditions; nothing else was altered between the members of a pair.

Duration: full results

Table A1

Optically measured duration of a 12-frame stimulus, photodiode on the screen, 50 % criterion on both edges. All rows but the last two used the same Dell 1905FP panel. "Off by >1 ms" counts trials outside a millisecond of the median; those are dropped or held frames, not scatter. The SD is given because it is conventional, but for these distributions it describes neither the bulk nor the tail — compare the p5–p95 column beside it, and see Figure 2.

Machine	Stack	n	Median	p5–p95	SD	Off by >1 ms
5490	Wayland	987	202.620 ms	0.08 ms	3.654 ms	**34 (3.4 %)**, to 236 ms
5490	kmsdrm	1004	203.150 ms	0.26 ms	0.091 ms	0
W5700	X11	476	197.250 ms	0.03 ms	0.011 ms	0
W5700	kmsdrm[a]	280	198.040 ms	0.79 ms	0.322 ms	0
5490	Wayland[b]	1005	221.050 ms	0.14 ms	2.648 ms	23 (2.3 %)
5490	kmsdrm[b]	1005	220.770 ms	0.15 ms	0.048 ms	0

[a] measured before the panel had settled: its 10–90 % rise time climbed 0.6 ms across the run, against 0.04 ms in the settled sessions, and the spread is that monotonic ramp rather than scatter (*Why the configurations differ*). [b] the laptop's built-in panel rather than the shared 1905FP.

What the panel contributes, and why absolute durations do not compare

The median duration is not 200 ms on any panel: it is 197.25 ms and 203.15 ms on *the same* Dell 1905FP driven by two different machines, and 220.77 ms on the laptop's built-in display. The framework presented exactly twelve frames in every case, and the two 1905FP sessions agree on the trigger-to-trigger interval to three decimals (499.830 ms), so no frame differed between them.

The difference is the panel. A liquid-crystal display does not rise and fall symmetrically, so a fixed-level criterion applied to both edges does not return the presented duration; the bias is approximately (fall − rise)/2. In the two 1905FP sessions the same monitor, at indistinguishable drive levels (photodiode swing 7.93 against 8.03 V, ruling out a difference in signal range), rose (10–90 %) in 16.8 and 10.1 ms and fell (90–10 %) in 8.2 and 8.9 ms, predicting biases of −4.4 and −0.6 ms and so accounting for most of the 5.9 ms between them; the remainder is within what these corrections resolve. The laptop panel rises in 6.1 ms, falls in 12.1 ms, and holds its last frame about 5 ms past the offset before decaying—an asymmetry of +6.0 ms, in the other direction.

The slower of the two 1905FP sessions was also the one still warming: its rise time climbed 0.6 ms across the run while the other held to 0.04 ms. `cmd/ad3-to-events.py` prints rise time, fall time, the implied criterion bias and a settling check for every capture, so that a cross-machine comparison can be corrected rather than believed.

The consequence for practice is not that the measurement is wrong but that "stimulus duration" names two quantities. The number an experiment controls is the frame count, and it is exact. The number a participant's retina integrates is panel-dependent and cannot be recovered from software. A duration reported without the panel and the criterion is not reproducible even on the same rig.

Onset timing: full results

Table A2

Onset timing, 1010 cycles per run except where noted. Trigger-to-light is measured on one instrument timebase; flip-to-photons additionally uses the run's own timestamps, with the host–instrument clock ratio fitted from the trigger-versus-flip relation over the whole run, so that a constant trigger latency cancels. De-trended SD is the scatter after removing any linear ramp; where the two SD columns differ, the raw one is describing a drift rather than a jitter.

Machine	Stack (trigger)	TTL→light	SD	Flip→photons	de-trended SD	drift
Pi 4	kmsdrm (GPIO)	23.43 ms	0.075 ms	—	0.128 ms	+0.01 ppm
W5700	X11 (DLP-IO8)	54.88 ms	0.296 ms	54.85 ms	0.278 ms	+0.13 ppm
W5700	kmsdrm (DLP-IO8)[a]	22.55 ms	0.057 ms	22.61 ms	0.028 ms	+1.20 ppm
5490	Wayland (DLP-IO8)	24.89 ms	0.850 ms	25.25 ms	0.674 ms	+2.05 ppm
5490	kmsdrm (DLP-IO8)	22.26 ms	0.137 ms	**22.28 ms**	**0.079 ms**	−0.77 ppm

[a] 286 cycles over 2.3 min, on a panel still settling; its drift is that transient, not a property of the stack.

The trigger devices cost microseconds. Subtracting trigger-to-light from flip-to-photons on the same run isolates the latency of the trigger device itself: +17 μs for the DLP-IO8 behind a USB-C hub on the 5490, and +12 μs for the same box behind a DisplayPort adapter on the W5700. A USB-serial trigger is often assumed to be the weak link in such a rig; here it is three orders of magnitude below the display stack's contribution.

Why the configurations differ

The display stack. Two machines were measured with the display server as the only variable. On the W5700, stopping the X server moved the trigger-to-light lag from 54.88 to 22.55 ms—32.4 ms, almost exactly two frames, consistent with an X11 Present path plus a driver swapchain queueing ahead of scanout. On the 5490 against a Wayland compositor the latency cost was smaller (3.0 ms) and the precision cost far larger: scatter 0.674 → 0.079 ms, dropped frames 15 → 0, and long stimuli 3.4 % → 0. Under Wayland only 32 consecutive clean cycles existed in 981 from which to fit a panel rate; under KMS/DRM all 1000 were usable.

Scaled modes on Wayland. With the 5490's desktop scaled to 1920×1200 on a 2560×1600 panel, SDL was told the mode was 1920×1200 at 59.95 Hz—with no scale factor declared—while the hardware ran at 60.0386 Hz. The reported refresh was wrong by **1477 ppm**, and 16.8 % of presents returned early enough to be held to a schedule built

on it. At native resolution the reported rate came within 10 ppm of the hardware and the held fraction fell to 0.2 %. This is the one case we found where the framework's own frame period was materially wrong through no fault of its own, and it cannot be detected from inside the program.

Panels settle, and it is worth waiting. Liquid-crystal response depends on temperature, so a panel measured soon after a mode change is still changing. A run begun seconds after a console switch showed its measured duration falling monotonically by 0.68 ms over 2.3 minutes, with no such ramp on a display that had been in the same mode for hours. The panel's *rate* moves too, by 23 ppm over six minutes while warming. Both argue for warming the display in its final mode before recording anything.

Audio in detail

Audio was characterised by capturing the line output electrically rather than through a microphone, which removes the acoustic path and the microphone's own onset criterion from the measurement.

The jitter is one buffer period, and it is a beat rather than noise. The tone is handed to the device on time—the software-side interval between the visual flip and the moment the tone was queued was 0.080 ± 0.035 ms—and then waits for the queue, so its onset scatters across one buffer period:

Buffer (frames at 48 kHz)	period	measured SD	period/$\sqrt{12}$	torn tones
256	5.33 ms	2.36 ms	1.54 ms	100 / 483
512	10.67 ms	3.34 ms	3.08 ms	10 / 500
1024	21.33 ms	6.17 ms	6.16 ms	0 / 481
2048	42.67 ms	12.27 ms	12.32 ms	0 / 60

Prediction and measurement agree to within 8 % at every point, and the same relation held on a USB interface on the other machine. The scatter is not random: the tone's position in the buffer grid advances by (cycle mod period) each trial and wraps. On the Pi, with a 499.830 ms cycle and a 10.667 ms buffer, that predicts +1.503 ms per trial with a −9.163 ms wrap; the measured values were +1.500 and −9.170 ms. A cycle chosen as a whole multiple of the buffer period removes the beat altogether—an experimenter who needs a fixed audio-visual lag can have one.

Below 1024 frames the audio tears. 2.0 % of tones at 512 frames and 20.7 % at 256 contained audible silences. At 512 the audio server responded to repeated underruns by adding a buffer period mid-run, stepping the latency by +10.85 ms—so the setting is not one that machine holds. The floor is the hardware and not the server: with the server stopped and the device opened directly through ALSA, every one of 463 tones was torn.

Availability

The measurement suite is `tests/Timing-Tests/` in the repository, and the analysis tools used here are `cmd/ad3-to-events.py`, `cmd/extract-av-sync.py`, `cmd/plot-durations.py` and `cmd/plot-timing-figures.py`. Capture is performed with `ad3-capture` (https://github.com/chrplr/ad3-capture), which is released separately because it is useful independently of this framework. Every run writes its full configuration—display mode, audio device and buffer, scheduling policy, onset source, and the pacing

branch each frame took—into a companion `-info.txt`, and the figures in this paper are regenerated from the raw captures by a single command.

```
# --EXPERIMENT INFO
# e mainfile: Timing-Tests
# e cmdline: Timing-Tests -test display -duration-s 3 -w
# e start_time: 20260818-142047
# e hostname: is158520
# [...]
# e os: linux/amd64
# e framework: goxpyriment (devel) --- see http://chrplr.github.io/goxpyriment
# [...]
# --SUBJECT INFO
# s id: 0
# [...]
# --SYSTEM INFO
# sys sdl_version: 3.4.0
# sys video_driver: wayland
# sys renderer: opengl
# sys gl_renderer: Mesa Intel(R) Arc(tm) Graphics (MTL)
# [...]
# sys fullscreen: false
# sys vsync: 1
# sys vblank_backend: not requested (set GOXPY_VBLANK=on to anchor onsets on kernel vblanks)
# sys refresh_nominal_hz: 60.0190
# sys refresh_measured_hz: 60.0025
# sys audio_driver: pipewire
# sys audio_device: Meteor Lake-P HD Audio Controller Speaker
# [...]
# sys audio_buffer_frames: 1024
# sys sched_policy: SCHED_FIFO
# sys sched_realtime: true
# sys sched_priority: 50
# [...]
# --HOST INFO
# host machine: Dell Inc. Precision 5490
# host machine_type: laptop
# host os: Ubuntu 26.04 LTS
# host kernel: 7.0.0-29-generic x86_64
# host desktop: ubuntu:GNOME
# host cpu: Intel(R) Core(TM) Ultra 7 165H
# host cpu_topology: 16 cores / 22 threads
# host cpu_mhz: 1667 (max 4700)
# host ram: 30.04 GiB (40% used at start)
# host gpu0: Intel Corporation Meteor Lake-P [Intel Arc Graphics] [driver i915]
# host gpu1: NVIDIA Corporation AD107GLM [RTX 2000 Ada Generation Laptop GPU] [driver nvidia]
# host audio_card: sof-soundwire [driver snd_soc_sof_sdw]
# host audio_server: PipeWire 1.6.2
# host alsa: k7.0.0-29-generic
# --DISPLAY INFO
# d id: 3
# d name: Dell Inc. 19"
# d native_resolution: 1280x1024
# [...]
# d refresh_rate_hz: 60.0200
# [...]
# d bounds: 0,0 1280x1024
# gc=suspended
# [...]
# test=display duration-s=3.0 warmup=10
# [...]
# sys pacing: presents=171 blocked=171 early=93 paced=0 vblank_held=0 (0.0 % paced)
# wait_mean=0.000 ms wait_max=0.000 ms frame=16.661 ms hold_frac=0.000 class=blocking
# e end_time: 20260818-142052.248
# e duration: 00:00:04.509
```

Figure A2

The companion `-info.txt` *written beside every session's CSV, from a short run on the laptop of Table A1 (abridged; each* `[...]` *marks omitted lines of the same kind, nothing is reworded).* `sys` *lines are what SDL reports about what it opened;* `host` *lines are what the operating system says about the machine. The CSV beside it carries no comment lines at all, so it loads into a spreadsheet or statistics package unmodified. Every field here is written without the experimenter doing anything, which is the point: the compositor, the sound server version and the scheduling policy obtained are all conditions that changed our own results by more than the experiment code did, and none of them can be recovered from the data afterwards.*